

ЛЕКЦІЯ 14

НЕЙРОННІ МЕРЕЖІ-3

1. ГРАФОВІ НЕЙРОННІ МЕРЕЖІ (GNN – GRAPH NEURAL NETWORKS)

2. ТРАНСФОРМЕРИ (TRANSFORMERS)

3. АВТОКОДЕРИ (AUTOENCODERS)

4. ГЕНЕРАТИВНО-ЗМАГАЛЬНІ МЕРЕЖІ (GANS – GENERATIVE ADVERSARIAL NETWORKS)

5. АНАЛІЗ НАУКОВОЇ СТАТТІ

1. ГРАФОВІ НЕЙРОННІ МЕРЕЖІ (GNN – GRAPH NEURAL NETWORKS)

- **Опис:** Працюють з графовими структурами даних.
- **Застосування:** *Аналіз соціальних мереж, молекулярне моделювання, маршрутизація.*

Графові нейронні мережі (**Graph Neural Networks, GNNs**) дозволяють працювати з графовими структурами даних, такими як соціальні мережі, молекулярні графи, дорожні мережі тощо. Одним із популярних фреймворків для реалізації GNN є PyTorch Geometric. Ось приклад програми, що демонструє використання GNN для задачі класифікації вузлів графа.

[CMM LEC 12-1]

#Проста GNN (двошарова GCN) на питру з візуалізацією.

- Створює маленький синтетичний граф з 2 кластерами

- Генерує прості фічі та мітки (2 класи)

- Реалізує forward/backprop вручну для двошарової GCN:

$$H = \text{ReLU}(\hat{A} X W_0)$$

$$Z = \hat{A} H W_1$$

- Навчає з крос-ентропією (градієнтний спуск)

- Малює: початкові фічі, вбудування (hidden) до/після навчання, криву втрат

"""

```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
```

```
np.random.seed(1)
```

```
def normalize_adj(A):
```

```
    """Побудова нормалізованої матриці  $\hat{A} = D^{-1/2} (A + I) D^{-1/2}$ """
```

```
    A_hat = A + np.eye(A.shape[0])
```

```
    deg = A_hat.sum(axis=1)
```

```
    D_inv_sqrt = np.diag(1.0 / np.sqrt(deg))
```

```
    return D_inv_sqrt @ A_hat @ D_inv_sqrt
```

```
def softmax_rows(Z):
```

```
    e = np.exp(Z - Z.max(axis=1, keepdims=True))
```

```
    return e / e.sum(axis=1, keepdims=True)
```

```
def cross_entropy_loss(P, Y_onehot):
```

```
    # average loss
```

```
    return -np.sum(Y_onehot * np.log(np.clip(P, 1e-12, 1.0))) / P.shape[0]
```

```
def accuracy(P, y):
```

```

    preds = np.argmax(P, axis=1)
    return (preds == y).mean()

# --- Create toy graph with two clusters ---
N1 = 10
N2 = 10
N = N1 + N2

# create two Erdos-Renyi clusters and connect them
lightly
G1 = nx.erdos_renyi_graph(N1, 0.6, seed=2)
G2 = nx.erdos_renyi_graph(N2, 0.6, seed=3)
G = nx.disjoint_union(G1, G2)
# add a few edges between clusters
G.add_edge(2, N1 + 1)
G.add_edge(5, N1 + 5)
G.add_edge(0, N1 + 9)

A = nx.to_numpy_array(G) # adjacency
A_norm = normalize_adj(A)

# node features: 2D features: nodes in cluster 1
around (1,0), cluster 2 around (0,1)
X = np.zeros((N, 2))
X[:N1] = np.random.randn(N1, 2) * 0.2 +
np.array([1.0, 0.0])
X[N1:] = np.random.randn(N2, 2) * 0.2 +
np.array([0.0, 1.0])

# labels: 0 for first cluster, 1 for second
y = np.array([0] * N1 + [1] * N2)
C = 2 # classes

# visualize initial graph positions
pos = nx.spring_layout(G, seed=4) # for plotting
graph structure

# --- GCN parameters ---
input_dim = X.shape[1]
hidden_dim = 2 # choose 2 so мы можем

```

візуалізувати приховані вбудування

```
W0 = 0.5 * np.random.randn(input_dim, hidden_dim)
W1 = 0.5 * np.random.randn(hidden_dim, C)
```

```
lr = 0.5
epochs = 200
```

one-hot labels

```
Y_onehot = np.zeros((N, C))
Y_onehot[np.arange(N), y] = 1.0
```

```
loss_history = []
acc_history = []
```

before-training embedding for comparison

```
H_before = np.maximum(0, A_norm @ X @ W0)
```

--- Training loop (manual backprop) ---

```
for epoch in range(epochs):
```

forward

```
Z1 = A_norm @ X @ W0          # (N, hidden_dim)
H = np.maximum(0, Z1)         # ReLU
Z2 = A_norm @ H @ W1          # (N, C)
P = softmax_rows(Z2)
```

```
loss = cross_entropy_loss(P, Y_onehot)
acc = accuracy(P, y)
```

```
loss_history.append(loss)
acc_history.append(acc)
```

gradients (manual)

$dZ2 = (P - Y) / N$

```
dZ2 = (P - Y_onehot) / N      # (N, C)
```

W1 gradient:

$Z2 = (A_norm @ H) @ W1 \rightarrow grad_W1 = (A_norm @ H).T @ dZ2$

```
AH = A_norm @ H
gradW1 = AH.T @ dZ2           # (hidden_dim, C)
```

```

# propagate back to H:
# dH_part = A_norm.T @ (dZ2 @ W1.T) ; A_norm
symmetric so A_norm.T = A_norm
dH = A_norm @ (dZ2 @ W1.T) # (N, hidden_dim)

# through ReLU:
ReLU_mask = (Z1 > 0).astype(float)
dZ1 = dH * ReLU_mask # (N, hidden_dim)

# grad W0:
# Z1 = (A_norm @ X) @ W0 -> gradW0 = (A_norm @
X).T @ dZ1
AX = A_norm @ X
gradW0 = AX.T @ dZ1 # (input_dim,
hidden_dim)

# gradient descent update
W0 -= lr * gradW0
W1 -= lr * gradW1

if (epoch + 1) % 50 == 0 or epoch == 0:
    print(f"Epoch {epoch+1:3d} Loss
{loss:.4f} Acc {acc*100:.1f}%")

# after-training embedding
H_after = np.maximum(0, A_norm @ X @ W0)
Z2_after = A_norm @ H_after @ W1
P_after = softmax_rows(Z2_after)
final_acc = accuracy(P_after, y)
print(f"\nFinal accuracy: {final_acc*100:.2f}%
Loss: {loss_history[-1]:.4f}")

# --- Plots ---
plt.figure(figsize=(15, 4))

# 1) initial node features (scatter)
plt.subplot(1, 3, 1)
plt.title("Початкові фічі (X)")
plt.scatter(X[:N1, 0], X[:N1, 1], label="class 0",
marker='o')
plt.scatter(X[N1:, 0], X[N1:, 1], label="class 1",

```

```

marker='s')
plt.xlabel("x1")
plt.ylabel("x2")
plt.legend()
plt.grid(True)

# 2) hidden embedding before/after training drawn
on graph layout
plt.subplot(1, 3, 2)
plt.title("ВБудування вузлів (прихований шар).
До/Після навчання")
h1 = H_before
h2 = H_after
# plot before (faint) and after (solid)
# before
plt.scatter(h1[:, 0], h1[:, 1], alpha=0.3, s=80)
for i in range(N):
    if y[i] == 0:
        plt.text(h1[i, 0], h1[i, 1], str(i),
        fontsize=8, alpha=0.3)
# after
plt.scatter(h2[:N1, 0], h2[:N1, 1], label="class 0
(after)", marker='o', s=90)
plt.scatter(h2[N1:, 0], h2[N1:, 1], label="class 1
(after)", marker='s', s=90)

plt.xlabel("h1")
plt.ylabel("h2")
plt.legend()
plt.grid(True)

# 3) loss curve
plt.subplot(1, 3, 3)
plt.title("Крива втрат")
plt.plot(loss_history)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)

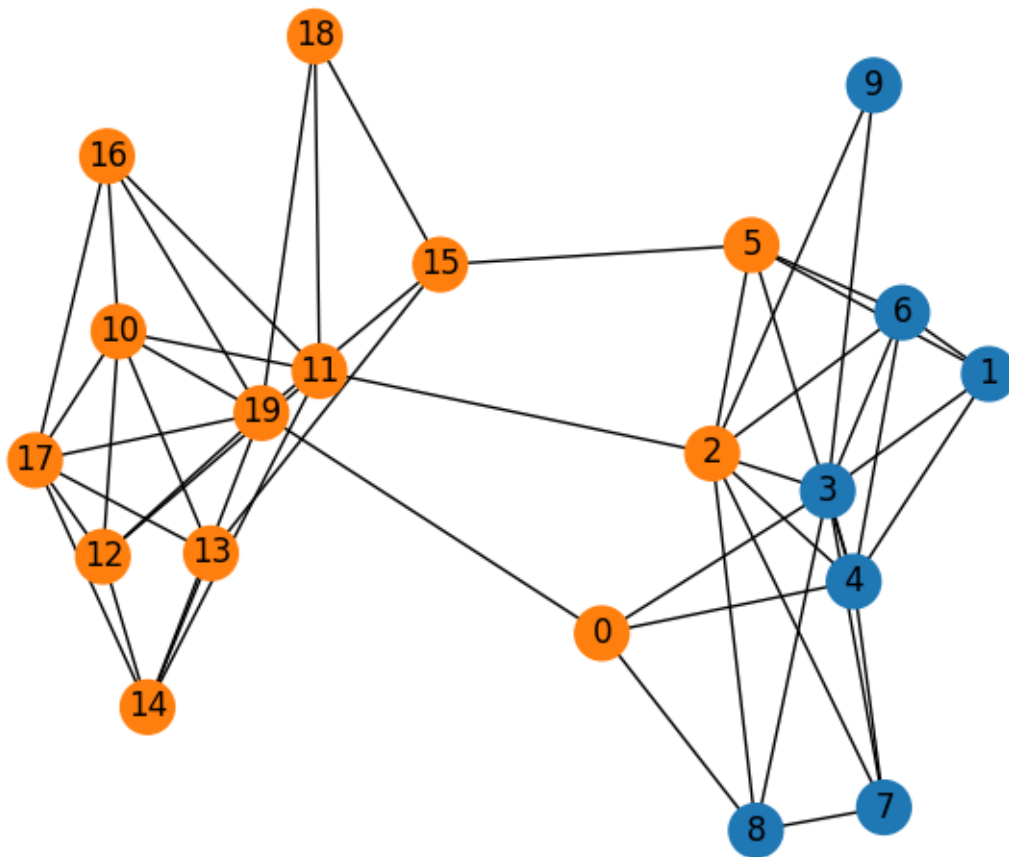
plt.tight_layout()
plt.show()

```

```

# Допоміжна візуалізація: реальна структура графа,
# кольори – передбачення
plt.figure(figsize=(6, 5))
preds = np.argmax(P_after, axis=1)
color_map = ['C0' if p == 0 else 'C1' for p in
preds]
nx.draw(G, pos, with_labels=True,
node_color=color_map, node_size=400)
plt.title("Структура графа з передбаченнями
(кольори вузлів)")
plt.show()

```



Python-програма (без PyTorch / PyG) яка ілюструє просту GNN (Graph Convolutional Network), навчає її на маленькому синтетичному графі для задачі класифікації вузлів і будує **графіки**: розсіяння вузлів у 2D (вбудування) до й після навчання та графік втрат.

побудована проста GCN-подібна модель, де множення на нормалізовану матрицю суміжності ($\hat{A}$) — механізм агрегації повідомлень між сусідами.

Модель двошарова: лінійний шар $\rightarrow$ ReLU $\rightarrow$ лінійний шар $\rightarrow$ softmax.

Навчання — простий градієнтний спуск.

`hidden_dim=2` дозволяє візуалізувати приховане вбудування вузлів у 2D — бачимо, як вузли кластеризуються після навчання.

Уявімо, що кожен вузол — це людина, а ребра — знайомства.

GNN будує **карту**, на якій:

- люди з однаковими інтересами і схожими друзями *наближаються*;
- різні групи людей *розходяться*.

Ця «карта» — і є **вбудування вузлів у 2D**.

До навчання — точки перемішані.

Після навчання — точки двох класів чітко розділені.

GNN навчилася «витягнути» інформацію з графа і представити її у вигляді зручних координат.

2. ТРАНСФОРМЕРИ (TRANSFORMERS)

Трансформери — це тип нейронних мереж, який у 2017 році радикально змінив галузь *обробки природних мов* (NLP) і надалі — комп'ютерний зір, звук, робототехніку.

Основна ідея трансформера: **повністю замінити рекурентність (RNN/LSTM) механізмом самоуваги (Self-Attention)**, що дозволяє паралельно опрацьовувати дані та ефективно розпізнавати залежності на великих дистанціях. Використовують механізм уваги (attention) для роботи з послідовностями. Є основою багатьох сучасних моделей, як-от GPT, BERT.

Застосування: Обробка природної мови, створення тексту, переклад.

[Архітектура Transformer — графічне представлення](#)

Недоліки попередніх моделей (RNN, LSTM, CNN у NLP)

RNN / LSTM

- Працюють послідовно → немає паралелізму.
- Важко зберігати довгі залежності (long-term dependencies).
- Висока обчислювальна складність.

CNN у NLP

- Вимагають великих ядер для «покриття» довгих контекстів.
- Не враховують порядок слів природно.

Трансформери усувають всі ці обмеження.

Архітектура трансформера

Трансформер складається з двох частин:

1. **Encoder (кодувальник)**
2. **Decoder (декодувальник)**

(у деяких моделях використовується тільки Encoder — BERT, або тільки Decoder — GPT).

Encoder

Кілька однакових шарів, і кожен містить:

- Self-Attention

- Feed-Forward Network
- Нормалізація + резидуальні зв'язки

Decoder

Містить:

- Masked Self-Attention
- Attention до виходу Encoder (Cross-Attention)
- Feed-Forward
- Нормалізація та резидуальні зв'язки

Механізм уваги (Attention)

Увага дозволяє моделі **визначати важливість кожного слова відносно інших.**

Основна формула (Scaled Dot-Product Attention)

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

Де:

- **Q** — запити (queries)
- **K** — ключі (keys)
- **V** — значення (values)

Multi-Head Attention

Кілька незалежних механізмів уваги працюють паралельно.

Це дозволяє «бачити» різні типи залежностей.

Позиційне кодування (Positional Encoding)

Оскільки в трансформері немає рекурентності, порядок подається штучно:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Позиційне кодування *додається до векторів слів*.

Типи моделей-трансформерів

1. Encoder-only (бідирекційні)

- BERT
- RoBERTa
- ALBERT

Використання: класифікація, пошук інформації, *аналіз тональності*.

2. Decoder-only (авторегресивні)

- GPT, GPT-2, GPT-3, GPT-4, GPT-5

Використання: генерація тексту, переклад, чат-боти.

3. Encoder–Decoder (seq2seq)

- T5
- BART

Використання: переклад, узагальнення текстів.

Переваги трансформерів

Паралельність

Можна навчати моделі на GPU тисячами токенів одночасно.

Довгий контекст

Увага дозволяє враховувати залежності між далекими словами.

Масштабованість

Ефективно ростуть при збільшенні параметрів (закон «scaling laws»).

Універсальність

Працюють не лише з текстом, а й:

- зображення (Vision Transformers, ViT)
- аудіо (Whisper)
- біологія (AlphaFold)
- мультиагентні системи
- робототехніка

Недоліки трансформерів

- Висока обчислювальна складність ($O(n^2)$ через self-attention).
- Великі вимоги до пам'яті.
- Потребують величезних датасетів.

Оптимізовані моделі трансформерів

Для зменшення складності розроблено варіанти:

- **Linformer**
- **Longformer**
- **Performer**
- **Reformer**
- **FlashAttention**
- **Mistral MoE**

- **LLaMA-3/4**

Вони знижують обчислювальну вартість до $O(n)$ або $O(n \log n)$.

Застосування трансформерів

У NLP

- переклад
- чат-боти
- відповідь на запити
- генерація текстів
- пошук інформації
- резюмування

У зображеннях

- класифікація
- сегментація
- детекція (DETR)

У науці

- моделювання білків (AlphaFold)
- моделювання клімату
- моделювання хімії та матеріалів

Загальний алгоритм роботи трансформера

1. Текст $\rightarrow$ токени
2. Токени $\rightarrow$ вектори (*embedding*)
3. Додається positional encoding
4. Проходження через шари Self-Attention

5. Feed-Forward

6. Генерація або класифікація

Підсумки

Трансформери — це найпотужніша сучасна архітектура нейронних мереж, яка:

- забезпечує паралельне навчання,
- добре працює на довгих послідовностях,
- масштабована до мільярдів параметрів,
- застосовується в багатьох галузях: від тексту до біології.

3. АВТОКОДЕРИ (AUTOENCODERS)

- **Опис:** Мережі, які стискають дані (encoding) у компактне представлення, а потім відновлюють їх (decoding).
- **Застосування:** Зменшення розмірності даних, виявлення аномалій.

Автокодери — це тип нейронних мереж, які **навчаються відтворювати вхідні дані**, стискаючи їх у компактне представлення (латентний простір), а потім відновлюючи назад.

Простими словами:

- На вході: зображення, текст, сигнал, будь-які дані.
- У середині: стискання → найбільш важлива інформація.
- На виході: відновлення тих самих даних.
- **Ціль: мінімізувати різницю між входом і виходом.**

[\[перейти до папки лекції\]](#)

1. Архітектура автокодера

Автокодер складається з трьох частин:

1.1. Encoder (кодувальник)

Перетворює вхідні дані у вектор низької розмірності — **latent vector (z)**.

Задача: *максимально стиснути інформацію*.

Формально:

$$z = f_{\theta}(x)$$

1.2. Latent space (прихований простір)

Вектор (зазвичай 10–512 чисел), що містить найважливіші особливості даних.

1.3. Decoder (декодувальник)

Відновлює дані з латентного вектора:

$$\hat{x} = g_{\phi}(z)$$

Мережа навчається *мінімізувати помилку*:

$$L = \|x - \hat{x}\|^2 \text{ або BCE}$$

2. Основні типи автокодерів

2.1. Класичний автокодер

Просто навчається стисненню.

Недоліки: може запам'ятовувати дані, латентний простір без структури.

2.2. Denoising Autoencoder (DAE)

Навчається відновлювати **пошкоджені** дані.

Корисний для шумозниження:

- шум на зображеннях
- відновлення сигналів
- фільтрація шуму в даних датчиків

2.3. Sparse Autoencoder

Додається обмеження sparsity → латентний вектор має бути розрідженим.

Це допомагає формувати *важливі* ознаки.

2.4. Contractive Autoencoder

Мінімізує чутливість до малих змін — *стабільний до шумів*.

Дає “гладкий” латентний простір.

2.5. Convolutional Autoencoder (CAE)

Використовує згорткові шари → дуже добрий для зображень.

Застосування:

- шумозниження
- пошук аномалій у відео
- генерація нових зображень (в парі з GAN)

2.6. Variational Autoencoder (VAE)

Найважливіший сучасний різновид.

Відмінність:

Замість фіксованого вектора $z \rightarrow$ модель навчається *розподілу*

$$z \sim N(\mu, \sigma)$$

Переваги:

- можна генерувати нові дані (як у GAN)
- гладкий латентний простір
- можливість інтерполяції між об'єктами

VAE застосовуються у:

- генерації зображень
- стисненні
- anomaly detection
- біоінформатиці
- генерації музики/мови

2.7. Seq2Seq Autoencoders

Для тексту та сигналів.

Архітектури:

- **LSTM autoencoders**
- **GRU autoencoders**
- **Transformers autoencoders** (сучасний варіант)

3. Навіщо потрібні автокодери?

3.1. Стиснення даних (компресія)

Наприклад, стискання зображень краще за JPEG у спеціальних задачах.

3.2. Виділення ознак (feature learning)

Заміняють PCA, але набагато потужніші.

3.3. Виявлення аномалій

Логіка проста:

- автокодер добре відновлює нормальні дані
- погано відновлює аномальні

Застосування:

- технічна діагностика
- аналіз сенсорних мереж
- кібербезпека
- медичні дані

3.4. Генерація зображень (VAE)

Можна створювати “нові” картинки того ж класу.

3.5. Шумозниження

Згладжування зображень, сигналів, аудіо.

3.6. Попереднє навчання для інших моделей

Особливо для глибоких моделей, коли немає великих датасетів.

Переваги та недоліки

Переваги

- гнучкі
- працюють з будь-якими типами даних
- дають глибоке представлення (feature learning)
- можуть генерувати нові дані (VAE)

Недоліки

- можуть запам'ятовувати замість узагальнення
- латентний простір автокодера без VAE неконтрольований
- генерація у звичайних автокодерах слабша ніж у GAN
- потребують багато даних

7. Автокодери в реальних системах

- виявлення несправностей у промисловості
- аналіз медичних знімків

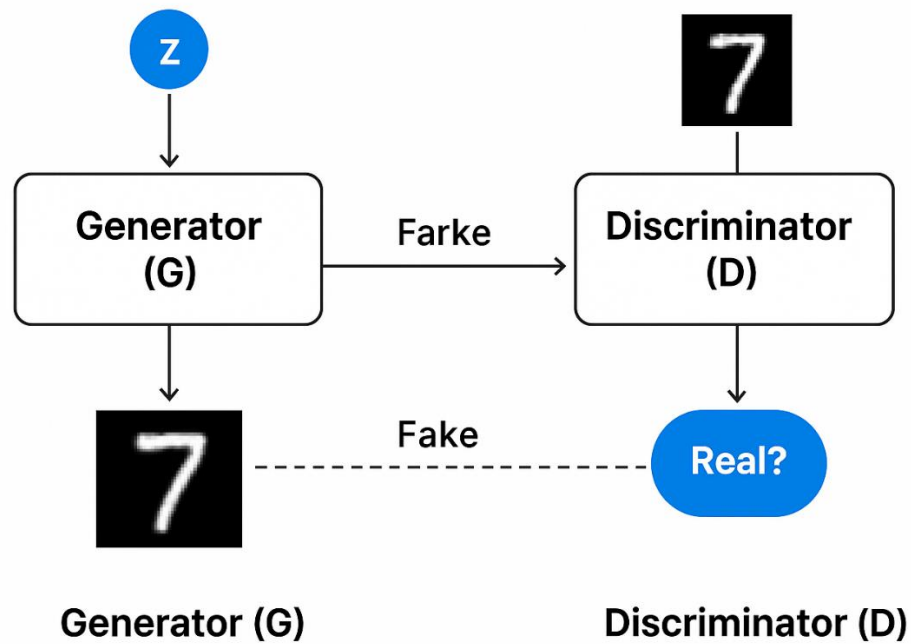
- *стиснення відео у режимі реального часу*
- *відновлення сигналів з датчиків дронів*
- *anomaly detection у фінансах*
- *попереднє навчання у NLP-моделях (Seq2Seq)*
- *відновлення пошкоджених фотографій*

4. ГЕНЕРАТИВНО-ЗМАГАЛЬНІ МЕРЕЖІ (GANS – GENERATIVE ADVERSARIAL NETWORKS)

- **Опис:** *Складаються з двох мереж: генератора і дискримінатора, які змагаються між собою. Генератор створює дані, дискримінатор оцінює їхню якість.*
- **Застосування:** *Генерація зображень, відео, аудіо, тексту.*

Generative Adversarial Networks — це клас нейронних мереж, запропонований Ієном Гудфеллоу (Ian Goodfellow) у 2014 році, який дозволяє **генерувати нові дані**, дуже схожі на реальні приклади.

Простими словами: GAN — це «штучний художник», якого навчає «штучний критик».



1. Суть і принцип роботи GAN

GAN складається з *двох* нейронних мереж, які тренуються *разом*, але мають *протилежні цілі*:

1. Generator (G)

- Приймає випадковий шум (зазвичай вектор $z \sim N(0,1)$)
- Генерує зображення або дані
- Його задача: **обдурити дискримінатор**

2. Discriminator (D)

- Приймає зображення (справжні або згенеровані)
- Визначає: *реальне чи фейкове*
- Його задача: **не дати себе обдурити**

Як відбувається тренування

Мережі грають у гру “змагання”:

1. Дискримінатор вчиться розрізняти реальні дані й згенеровані.

2. Генератор вдосконалюється, щоб створювати все якісніші фейкові приклади.
3. Процес повторюється, доки **генератор не почне створювати дані, які майже не відрізняються від справжніх.**

2. Математична ідея

Функція втрат GAN — це *гра мінімум–максимум*:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{real}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]$$

- **D** максимізує здатність відрізняти реальне від фейкового.
- **G** мінімізує її, щоб D помилявся.

Це і є *змагальна тренування*.

3. Що можуть робити GAN?

GAN сьогодні — одна з найпотужніших генеративних технологій.

3.1. Генерація зображень

- фотореалістичні обличчя
- нові моделі автомобілів
- нові архітектурні стилі
- одяг, меблі, предмети дизайну

3.2. Створення deepfake

- заміна облич
- генерація відео

3.3. Розфарбовування зображень

- чорно-білі → кольорові

3.4. Покращення роздільної здатності

- **Super-resolution GAN (SRGAN)**
- покращує якість зображень у кілька разів

3.5. Генерація музики, голосу, шумів

3.6. Доповнення датасетів

- вирішення проблеми “мало даних”

3.7. Видалення шумів, артефактів

3.8. Генерація сенсорних даних для моделювання дронів

- дані LIDAR
- IR-камери
- карти траєкторій

4. Типи GAN (найважливіші)

4.1. Vanilla GAN

Оригінальна модель Гудфеллоу.

4.2. DCGAN (Deep Convolutional GAN)

Використовує CNN.

Класичний GAN для зображень.

4.3. WGAN (Wasserstein GAN)

Вирішує проблему нестабільного навчання GAN.

Переваги:

- стабільніший тренувальний процес
- краща якість зображень
- контрольована метрика

4.4. WGAN-GP

Ще стабільніший варіант, який сьогодні використовується найчастіше.

4.5. Conditional GAN (cGAN)

Генерація *об'єктів певного класу*.

Наприклад:

- генеруй «коти»
- генеруй «літаки»
- генеруй «дрон у камуфляжі»

4.6. CycleGAN

Перетворення зображень без пар:

- зима ↔ літо
- фото дня ↔ фото ночі
- стилізація художників (ван Гог ↔ Пікассо)

4.7. StyleGAN / StyleGAN2 / StyleGAN3

Найкращий GAN у світі для генерації облич.

Використовується у deepfake, цифрових персонажах, 3D-моделях.

4.8. BigGAN

Надпотужна модель від Google: високоякісні зображення 512×512.

4.9. Pix2Pix (paired)

Перетворення *картинка* → *картинка*:

- контур → реалістичне зображення
- схема → будинок
- маска → об'єкт

5. Застосування GAN у науці та інженерії

У фізиці:

- моделювання розподілів частинок
- відтворення сигналів з детекторів
- симуляція рідкісних подій

В екології та логістиці:

- генерація карт траєкторій дронів
- синтетичні супутникові знімки
- симуляція потоків даних датчиків

У машинному навчанні:

- розширення датасетів
- симуляція даних для anomaly detection

- створення synthetic data для навчання нейромереж

У комп'ютерному зорі:

- стилізація
- суперроздільність
- inpainting (відновлення missing parts)

5. АНАЛІЗ НАУКОВОЇ СТАТТІ



Short-Term_Road_Speed_Forecasting_Base